

# API access

The client API, the port it answers on, the login that replaced the old one, and what a remote session may reach.

---

The client API is the reason this port exists. SD Core for Windows is built to be used as a back end data store reached through the API, and that is the tie-breaker on most other design questions here.

**The API is a normal way for any account to use SD**, not a facility reserved for developers and administrators. A person running a custom GUI program that talks to SD needs API access and may need nothing else — no ssh, no terminal, no development verbs. A standard-tier account with `api` access is an ordinary thing to create, and probably the commonest shape a deployed system will have:

```
create.account user jane api
```

Your application code does not change. `SDConnect()` and `SDConnectLocal()` take the same arguments and return the same things. **What changed is underneath: the login protocol, the port, the identity a session runs as, and what it is allowed to open.**

**The account's tier and its access route are independent.** Tier decides which verbs are in the VOC (Account types); `ssh` / `api` / `both` / `none` decides how the account is reached. A standard account with API access, a programmer with ssh only, and an administrator with both are all normal.

---

## The login is SCRAM-SHA-256, and the old one is gone

**A client that sends a password in clear is refused.** It gets *"Cleartext login is no longer supported; this server requires SCRAM authentication"* and the connection drops.

The API used to send the user name and password as plain text — anything able to watch the connection could read the password. It now runs a challenge and response: the server sets a puzzle only someone who knows the password can answer, and **the password itself is never sent in any form.**

**The server also proves itself to you.** It finishes by returning a value only the real server can compute, and the client refuses the connection if it does not match. Another program that grabbed the port before SD started cannot pretend to be SD in order to collect passwords.

---

**Use a client library from this release or later.**

An older one is refused outright

---

**Run `modify.password` again for every account that uses the API.**

The stored credentials changed shape and the old ones cannot be converted

---

**An account whose password has not been re-set is refused, and the refusal reads as a wrong password.** From the server's point of view there is no credential to check. If a working client suddenly cannot log in after an upgrade, this is the first thing to try.

The old credentials cannot be converted because **the password was never kept anywhere, by design** — there is nothing to convert them from.

**Programs using the `!sdclient` class are covered.** The class module BASIC programs use to reach another SD server speaks the new login too. Your code does not change — `connect()` takes the same arguments — but the program has to be running under this release **at both ends**.

`SDConnectLocal()` **connections are unaffected.** They send no password and never did.

---

## The port

`sd.conf` sets `APIPORT=4243`, and the server accepts API connections on any network interface. **Earlier builds of this port shipped it commented out and listened on `127.0.0.1` only**, which is why an ssh tunnel was needed.

**If you tunnel, stop.** `ssh -L 4243:127.0.0.1:4243 user@host` still works but is no longer what the design expects, and it is not tested. Point the client straight at port 4243 on the server.

**Reaching the port from another computer is off unless you tick the box during installation.** To change it afterwards, from an elevated prompt:

```
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Open
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Restrict
```

**To turn the API off altogether**, comment out the `APIPORT` line in `C:\ProgramData\SD\sd.conf` and restart SD. With no `APIPORT` set, SD creates no socket at all — "no API" is a real state, not just a firewall rule.

**Declining the API at install time leaves no listener at all.** The installer writes an `sd.conf` with no `APIPORT` line, so nothing listens and there is no firewall rule to open. The API box is unticked by default. `remote.api on` puts a listener back.

`APILOGIN` **is not an off switch.** It decides whether the API demands a password. `APILOGIN=0` is the **weaker** setting, not the safer one. Do not reach for it.

---

# Reaching the port is not getting in

A caller must clear three gates, in this order:

1. **Complete the SCRAM exchange** against a password held for that account — so **an account with no password cannot connect at all**.
2. **Be a member of the `sdapi` group**, which an account joins only when you give it API access. `create.account user fred api` or `... both` does that; `ssh` or `none` does not.
3. **If the account is an administrator, be connecting from this machine**. An administrator is refused an API session from any other computer, however good the password. Ordinary and programmer accounts are not affected.
4. **Pass the account's own group check**.

Gate 3 catches only the administrator tier, and it is not a setting — there is no keyword that turns it off. An administrator always has API access and it always stops at this computer:

An administrator may not sign in to this machine from another one.

**A local API client is unaffected**, which is the point: an application on the server connecting to `127.0.0.1` works as an administrator exactly as before. The reason the tier is treated this way is that an administrator's session can reach the operating system, so SD requires one Windows can show a consent prompt on — the console, or a remote desktop or remote-control product installed as a service. The Administrator set covers it in full.

**Failed API logins are written to the audit trail**, with the reason — an administrator refused this way is recorded with the address it came from. See Other hardening.

---

## A session is confined to its own account

**UNTIL 21 Aug 2026 A CLIENT CONNECTING OVER THE API COULD OPEN ANY FILE ON THIS MACHINE — INCLUDING THE FILE SD KEEPS PASSWORDS IN**. Holding one ordinary account's password was enough, and no administration command was needed. **If you have used a build older than that, treat the passwords of every account as having been reachable**.

---

|                      |                                                                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Still allowed</b> | everything inside its own account, and the shipped SDSYS files every account needs — messages, <code>syscom</code> , the dictionaries, <code>sd.voclib</code> . Ordinary programs are unaffected |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

|                          |                                                      |
|--------------------------|------------------------------------------------------|
| <b>No longer allowed</b> | opening, renaming, deleting or listing anything else |
|--------------------------|------------------------------------------------------|

---

**A refused `OPEN` takes the `ELSE` branch and `STATUS()` is 3035**, which means *not permitted* rather than *not found*. That distinction matters when you are debugging: 3035 is a containment refusal, not a missing file.

**A suspended account is refused here too, and deliberately says nothing about why.** `modify.account fred suspended` denies the API as well as ssh, and the message is the one this page already gives for an account that does not exist and for one you are not granted:

```
User not allowed in requested account
```

**All three answer identically on purpose**, so the API cannot be used to enumerate which accounts exist or what state they are in. If you are debugging a client that has suddenly stopped connecting, `list accounts` from an SD session is where the answer is — the `Tier` column will read `SUSPENDED`.

#### IF YOUR DATA LIVES OUTSIDE AN ACCOUNT

Name the directory in the `NETDIRS` setting in `C:\ProgramData\SD\sd.conf`, separating several with a semicolon. Nothing else needs changing. `config('NETDIRS')` prints what is in force.

**THE PASSWORD FILE, THE PROGRAM CATALOGUE AND THE ACCOUNT REGISTER ARE NEVER REACHABLE from an API session, and cannot be added to `NETDIRS`.**

---

## An API session runs as you

**Records an API session creates are owned by the account that logged in**, and the session reaches files with your access rather than the service's. If your account may not read something, the API session may not read it either.

This was not true before 24 Aug 2026, and the way it failed is worth knowing because it was invisible: taking on your Windows identity applied only to the one thread that did it, and starting a short-lived helper process during the switch quietly put the session back to the service's own identity — **with no error and nothing in the log** — before it had opened a single file. So the login half worked and the part you would notice did not: records came out owned by the system account.

**If your identity cannot be taken on, the login is now refused** rather than continued with the service's identity. A session that believes it is you while holding the service's rights is worse than one that never started.

**There is an alarm for the condition returning.** If a session ever believes it is you while Windows says it is not, the error log gets:

```
API IDENTITY LOST at record write - session believes it is ...
```

Nothing is written when the two agree, so **a healthy session is silent** and you should never see this. It refuses and slows nothing; it exists so the condition cannot go unnoticed a second time.

---

## `sh` and `OS.EXECUTE` are refused over the API

**Before this port they were not refused**, and on Windows that turned out to be worse than it sounds: **an API session ran as the LocalSystem account**, so a remote client could run any command on the machine with full privilege — more than the administrator sitting at the keyboard gets.

An API session is no longer treated as an administrator for any purpose, which is what SD's own code already assumed and did not enforce.

---

## Client libraries

---

|        |                           |
|--------|---------------------------|
| 64-bit | <code>sdclilib.dll</code> |
|--------|---------------------------|

---

|        |                                                                             |
|--------|-----------------------------------------------------------------------------|
| 32-bit | built separately; 32-bit remains a supported target, not a test convenience |
|--------|-----------------------------------------------------------------------------|

---

**Both must come from this release or later.** A client library that predates SCRAM is refused by the server, and the 32-bit client in particular once shipped sending passwords in clear — check which one your application is actually loading.